

Runbook & Troubleshooting

Operational procedures and known issues

Document	7 of 9
Audience	DevOps / On-call
Revision	Rev 1.0 · 2026
Status	Handoff

Castillo Planset QC — Runbook & Troubleshooting

Audience: DevOps / on-call engineers operating the Castillo Planset QC application on Azure Container Apps. **Scope:** step-by-step operational procedures with representative commands. All secret values in commands are shown as placeholders (`<...>`); never paste real keys, secrets, or connection strings into a shared terminal, a commit, or this document.

0. Environment reference

Two Azure Container Apps run the same image from the same registry; they differ only in app name, image tag, and (for the sandbox) scale settings.

	Production	Sandbox
Container App	<code>castillo-qaqc-automation</code>	<code>castillo-qaqc-sandbox</code>
Resource group	<code>castillo-qaqc-automation-rg</code>	<code>castillo-qaqc-automation-rg</code>
CI trigger branch	<code>master</code>	<code>staging</code>
Workflow	<code>.github/workflows/deploy.yml</code>	<code>.github/workflows/deploy-staging.yml</code>
Image tags pushed	<code>planset-qc:<sha></code> and <code>planset-qc:latest</code>	<code>planset-qc:staging-<sha></code>
Region	East US	East US

Shared infrastructure:

- **Container Registry (ACR):** `castilloqaqcautomationacr` — images pulled via a user-assigned managed identity (`castillo-qaqc-automation-id`), no registry password.
- **Auth:** Microsoft Entra built-in auth (EasyAuth), single-tenant, org login only. The app reads identity from the `X-MS-CLIENT-PRINCIPAL*` headers the sidecar injects (`backend/app/auth.py`).
- **Persistence:** SQLite on an Azure Files (SMB) share mounted at `/home/data` (env `PLANSET_DATA_DIR=/home/data`). The DB file is `/home/data/planset_qc.sqlite3`; PDFs, snippets, exports, and logs live in sibling folders (`runs/`, `exports/`, `logs/`).
- **Replica model:** **one** always-on replica (`minReplicas: 1`, `maxReplicas: 1`) in prod. SQLite is opened with `noLock=1` plus an in-process lock, which is correct only for a single writer. **Never raise** ``maxReplicas`` without first migrating off SQLite.

Common shell variables used throughout (PowerShell):

```
$RG      = "castillo-qaqc-automation-rg"
$APP     = "castillo-qaqc-automation"      # or castillo-qaqc-sandbox
$ACR     = "castilloqaqcautomationacr"
```

```
$STORAGE = "<storage-account-name>" # the Azure Files account (starts with "st")
$SHARE   = "data"
```

Key application environment variables (defined in the image / Bicep, overridable per Container App):

Var	Default	Purpose
AI_PROVIDER	openai	AI backend: <code>openai</code> or <code>gemini</code>
OPENAI_API_KEY	(secret)	OpenAI key; stored as Container App secret <code>openai-api-key</code>
OPENAI_MODEL	<code>gpt-5.4-mini</code> (prod)	Per-page vision model
OPENAI_MODEL_DEEP	<code>gpt-5.4</code> (prod)	Deep / cross-sheet reasoning model
OPENAI_SEED	(unset)	Best-effort determinism seed; unset = normal sampling
ANALYZE_CONCURRENCY	2	Max simultaneous analyses in the in-process queue
AI_TIMEOUT	90	Per-request AI timeout (seconds)
AI_RETRIES	2	Retries on 429/503/504
PLANSET_DATA_DIR	<code>/home/data</code>	Root of all persistent data
FRONTEND_DIST	<code>/app/frontend_dist</code>	Built SPA served by FastAPI
DEV_USER_EMAIL	(unset)	Local-dev identity fallback only; not used in cloud

1. Deploy to sandbox (push to staging)

Goal: roll a candidate build onto `castillo-qaqc-sandbox` without touching production.

1. Confirm your work is on (or merged into) the `staging` branch.
2. Push: ``powershell git push origin staging`
3. The `Deploy to Sandbox (staging)` workflow runs: OIDC login → `az acr build (tag planset-qc:staging-<sha>) → az containerapp update` on `castillo-qaqc-sandbox`.
4. Watch the run: ``powershell gh run list --workflow deploy-staging.yml --limit 5 gh run watch <run-id>`
5. Verify the new revision is live and healthy: ``powershell $URL = "https://$(az containerapp show -n castillo-qaqc-sandbox -g $RG --query properties.configuration.ingress.fqdn -o tsv)" curl "$URL/api/health" # -> {"ok":true}`

The sandbox may be scaled to zero (see §9). The first request after idle can cold-start for ~30–60 s — that is expected, not a failure.

2. Deploy to production (merge to master)

Goal: promote a reviewed change to `castillo-qaqc-automation`.

6. Merge the reviewed PR into `master` (do not push directly unless that is the agreed process).
```powershell git checkout master git pull origin master` Or via CI on merge — the `merge commit to master` is itself the trigger.
7. The `Deploy to Azure` workflow runs: `OIDC login` → `az acr build (tags planset-qc:<sha> and planset-qc:latest)` → `az containerapp update` on `castillo-qaqc-automation`. This creates a new revision and shifts 100% traffic to it.
8. Watch and verify: ```powershell gh run watch <run-id> $URL = "https://$(az containerapp show -n $APP -g $RG --query properties.configuration.ingress.fqdn -o tsv)" curl "$URL/api/health" # -> {"ok":true} curl "$URL/api/runs" # -> JSON array of runs`
9. **Rollback** (point the app at a previous image tag; no rebuild): ```powershell az containerapp update -n $APP -g $RG --image "$ACR.azurecr.io/planset-qc:<old-sha>"` List available image tags with `az acr repository show-tags -n $ACR --repository planset-qc --orderby time_desc -o table``.

The push-to-deploy workflow only rolls the **image** (`az containerapp update --image`). It never touches Entra auth, secrets, or the Azure Files mount — those are managed separately (§4, §5, §6).

## 3. Re-run a CI deploy after a Docker Hub rate-limit flake

**Symptom:** the `Build image in ACR` step fails while pulling a base image (`node:22-slim` or `python:3.11-slim`) with a `TOOMANYREQUESTS` / "You have reached your pull rate limit" message from Docker Hub. This is transient and not a code problem.

10. Confirm it is the rate limit, not a real build break: ```powershell gh run view <run-id> --log-failed` Look for `toomanyrequests / pull rate limit` on the base-image pull. If instead the app code failed to build, fix the code — do not just re-run.
11. Re-run only the failed job (no new commit needed): ```powershell gh run rerun <run-id> --failed` Or, from the Actions UI, use **Re-run failed jobs**.
12. If it keeps failing, trigger a fresh run manually via `workflow_dispatch` (both workflows enable it):  
```powershell gh workflow run deploy.yml # prod (master) gh workflow run deploy-staging.yml # sandbox (staging)`

13. As a last resort, build directly in ACR from your machine (ACR's build agents have their own pull allowance and usually succeed independent of Docker Hub's anonymous limit): ``powershell az acr build -r $ACR -t "planset-qc:<sha>" . az containerapp update -n $APP -g $RG -image "$ACR.azurecr.io/planset-qc:<sha>"`

On Windows, `az acr build`'s log streamer can crash on a Unicode character (UnicodeEncodeError: charmap) while the build still succeeds server-side. Confirm with `az acr task list-runs -r $ACR --top 1 -o table` and proceed when it shows **Succeeded**.

4. Trigger a re-analysis of an existing run

Goal: re-run QC on a planset that was already uploaded — for example after a rules/model change — without re-uploading the PDF. This is **non-destructive**: the existing run is preserved and the new result becomes the next *version* in the same lineage.

Preferred path — the UI:

14. Open the run, use **Re-run analysis**. The new job appears in the shared Activity panel as **queued**, then **running**, then **done**. The prior version stays current until the new one saves successfully.

API path (for scripted / on-call use):

15. Get the target `run_id` (from the URL, or `GET /api/runs`).
16. POST to the reanalyze endpoint (multipart form; `use_deep` defaults to `true`): ``powershell $URL = "https://$(az containerapp show -n $APP -g $RG --query properties.configuration.ingress.fqdn -o tsv)" curl -X POST "$URL/api/runs/<run_id>/reanalyze" -F "use_deep=true" # -> {"upload_id":"<uuid>","status":"queued","deep_mode":true,"parent_run_id":"<run_id>"}`

Behind Entra auth, browser-authenticated calls carry the signed-in identity automatically. Scripted calls from outside the browser session will be challenged by EasyAuth; run this from the authenticated UI or an authorized session rather than an anonymous curl.

17. Poll for completion: ``powershell curl "$URL/api/progress/<upload_id>" # {"step":..., "pct":...} until pct=100 / step=done curl "$URL/api/jobs" # shared activity feed`
18. On success the lineage's newest version is flagged `is_latest`; the previous versions remain queryable at `GET /api/runs/<run_id>/versions`. If the reanalysis **fails**, the prior version is left intact and current (the demotion only happens after a successful save).

5. Add or rotate the OpenAI API key

Symptom / goal: the key is being rotated on schedule, or analyses fail with 401/authentication errors from OpenAI. The key is a Container App **secret** named `openai-api-key`, surfaced to the app as `OPENAI_API_KEY`.

The person doing the rotation enters the key value themselves. Do not ask anyone to paste a real key into chat or into a shared/logged terminal.

19. Obtain the new key from the OpenAI account (out of band).
20. Set the secret (the operator types the value directly): `powershell az containerapp secret set -n $APP -g $RG --secrets openai-api-key=<new-openai-key>`
21. Confirm the env var still references the secret (it should already; only verify if in doubt):
`powershell az containerapp show -n $APP -g $RG --query "properties.template.containers[0].env[?name=='OPENAI_API_KEY']"`
22. Restart the current revision so the running process picks up the new value: `powershell $REV = az containerapp revision list -n $APP -g $RG --query "[?properties.active].name | [0]" -o tsv az containerapp revision restart -n $APP -g $RG --revision $REV`
23. Verify by starting a small analysis or checking logs (§9) for successful AI calls and absence of `OPENAI_API_KEY is not set / 401` errors.

6. Rotate the Entra (EasyAuth) client secret

Symptom / goal: the app registration's client secret is expiring or compromised, or sign-in fails with an authentication-configuration error. **Client secrets are write-only** — Azure never returns an existing secret's value, so rotation always means creating a **new** secret and wiring it in. The operator performing this enters/handles the value themselves; never surface it in chat.

24. Reset (recreate) the secret on the app registration used for EasyAuth. This prints the **new** value once — capture it securely: `powershell az ad app credential reset --id <authClientId> --query password -o tsv`
25. Re-apply the auth wiring by redeploying the infrastructure template, passing the new secret securely (it is defined as an `authConfigs` resource in `infra/main.bicep`, so a template deploy preserves the gate). The OpenAI key must also be passed since the same template owns it: `powershell az deployment group create -g $RG --template-file infra/main.bicep --parameters infra/main.parameters.json --parameters openAiApiKey=<openai-key> authClientSecret=<new-entra-secret>`
26. Verify sign-in end to end: open the app URL in a fresh private browser window, complete the Microsoft org login, and confirm you land on the app rather than an auth error.

The push-to-deploy CI never touches auth, so a normal deploy will **not** fix or break the auth secret — this template redeploy is the only path. `authClientId` and `authTenantId` are already in `infra/main.parameters.json`; only the secret is supplied at deploy time.

7. Snapshot / back up the SQLite database

Goal: take a consistent copy of `/home/data/planset_qc.sqlite3` before a risky migration, model change, or bulk edit.

Because the DB is opened with `no-lock=1` on an SMB share, a raw file copy taken mid-write can be torn. Prefer SQLite's online-backup API for a consistent snapshot, then pull the file off the share.

Option A — consistent snapshot via the backup API (recommended). Exec into the replica (§9) and use the bundled Python to write a coherent copy into the mounted share:

27. Exec in: ``powershell az containerapp exec -n $APP -g $RG --command /bin/sh``
28. Inside the container, snapshot to a timestamped file on the same share: ``sh python - <<'PY'``

```
import sqlite3, time
ts = time.strftime("%Y%m%d-%H%M%S")
src = sqlite3.connect("/home/data/planset_qc.sqlite3")
dst = sqlite3.connect(f"/home/data/backups/planset_qc-{ts}.sqlite3")
src.backup(dst);
dst.close(); src.close()
print("snapshot written")
PY`
```

(Create `/home/data/backups` first with `mkdir -p /home/data/backups` if it does not exist.)
29. Exit the container.

Option B — download the snapshot from the Azure Files share to your machine or to long-term storage:

```
az storage file download --account-name $STORAGE --share-name $SHARE `
--path "backups/planset_qc-<ts>.sqlite3" `
--dest "./planset_qc-<ts>.sqlite3" --auth-mode login
```

For a quick, lower-consistency copy you can download the live DB file directly (`--path planset_qc.sqlite3`), but only do this during a quiet period — there is no guarantee against an in-flight write. Option A is the safe default before migrations.

To restore, stop new activity, replace `/home/data/planset_qc.sqlite3` with the snapshot (via `az storage file upload` or from within an exec session), and restart the revision (§9). The app runs its idempotent migrations/backfill on startup (`init_db`).

8. Enable OPENAI_SEED (best-effort determinism) on a container

Goal: make repeated analyses of the same planset return near-identical findings — useful for reproducing a run or producing clean before/after regression diffs. This is best-effort per OpenAI: a model/infra change (new `system_fingerprint`) can still shift results, and it does not resolve genuine model-judgment differences on ambiguous drawings.

30. Set the env var (creates a new revision): ``powershell az containerapp update -n $APP -g $RG --set-env-vars OPENAI_SEED=<integer>`` Do this on the **sandbox** first when validating.
31. Confirm it took: ``powershell az containerapp show -n $APP -g $RG --query "properties.template.containers[0].env[?name=='OPENAI_SEED']"`
32. To disable and return to normal sampling, remove the variable: ``powershell az containerapp update -n $APP -g $RG --remove-env-vars OPENAI_SEED``

OPENAI_SEED is read at call time (`backend/app/gemini_client.py`). temperature is intentionally not exposed — the gpt-5.x reasoning models ignore or reject it.

9. Wake a scaled-to-zero sandbox, check logs, exec into the replica

Symptom / goal: the sandbox (or a `minReplicas: 0` deployment) shows zero replicas; you need to warm it, read logs, or get a shell.

33. **Wake it** by issuing any request — the ingress spins up a replica (cold start ~30–60 s):
``powershell $URL = "https://$(az containerapp show -n castillo-qaqc-sandbox -g $RG --query properties.configuration.ingress.fqdn -o tsv)" curl "$URL/api/health"`
34. **Confirm a replica is running:** ``powershell az containerapp replica list -n castillo-qaqc-sandbox -g $RG -o table``
35. **Console logs (streamed from the platform):** ``powershell az containerapp logs show -n $APP -g $RG --type console --tail 100 az containerapp logs show -n $APP -g $RG --type console --follow # live tail``
36. **Application log file** on the share (`RotatingFileHandler`, 5 MB × 5 files) — read via `exec`, or download it: ``powershell az storage file download --account-name $STORAGE --share-name $SHARE --path "logs/plansetqc.log" --dest ". /plansetqc.log" --auth-mode login``
37. **Exec into the running replica** for interactive inspection: ``powershell az containerapp exec -n $APP -g $RG --command /bin/sh # inside: ls -la /home/data ; tail -n 100 /home/data/logs/planset_qc.log``

10. Interpret a stuck or errored job in the queue

Background: every analysis and reanalysis is submitted to an in-process, bounded queue (`backend/app/jobs.py`) backed by a `ThreadPoolExecutor` of `ANALYZE_CONCURRENCY` workers (default

2). Job lifecycle is **queued** → **running** → **done** | **error**. The shared Activity panel reads **GET /api/jobs**; finished jobs are retained for **30 minutes** or the **last 40** finished jobs, whichever is smaller, then pruned from the feed.

Diagnose:

38. Read the shared feed: `powershell curl "$URL/api/jobs" # {"jobs":[...], "concurrency":2, "queued":<n>, "running":<n>}` Each job carries **status**, **queue_position** (1-based, only while queued), **pct**, **step**, **detail**, and (on failure) **error**.

Interpret by state:

Observation	Likely cause	Action
Job stays queued with a queue_position	All ANALYZE_CONCURRENCY slots busy; it will start when one frees	Wait; or reduce load. To raise throughput, bump ANALYZE_CONCURRENCY (\$below) — mind OpenAI rate limits and the single replica's 4 GiB.
Job running , pct not advancing for a long time	A slow or hung AI call, a very large PDF, or an AI timeout/retry loop	Check console logs and planset_qc.log for AI 429/503/504 ... retrying or a stack trace. AI_TIMEOUT (90 s) × AI_RETRIES (2) bounds each call.
Job error with an error message	fn() raised (e.g. OPENAI_API_KEY is not set , PDF parse failure, OpenAI 401/quota)	Read the error field and the logged Analysis failed for <upload_id> traceback; fix the root cause (key \$5, quota, corrupt PDF) and re-submit.
Job vanished from the feed	Finished > 30 min ago or pushed out beyond the 40-job retention window	Not an error — query the run itself via GET /api/runs/<run_id> / the versions endpoint.

Recovering a wedged queue: the registry and executor are **in-memory and single-replica**. If a worker thread is genuinely hung and blocking a slot, restart the active revision to reset the queue (this abandons in-flight jobs — they must be re-submitted):

```
$REV = az containerapp revision list -n $APP -g $RG --query "[?properties.active].name | [0]" -o tsv
az containerapp revision restart -n $APP -g $RG --revision $REV
```

Adjusting concurrency:

```
az containerapp update -n $APP -g $RG --set-env-vars ANALYZE_CONCURRENCY=<n> # new
revision
```

Keep this low: the single replica shares one OpenAI key and CPU, so too many parallel deep runs thrash memory and risk rate limits.

11. Known issues & gotchas

- **Single replica is load-bearing.** SQLite on SMB with `noLock=1` + in-process lock is safe for exactly one writer. Raising `maxReplicas` above 1 will corrupt data. Horizontal scale requires migrating the DB to Azure Database for PostgreSQL and artifacts to Blob Storage; the job queue would also need to move to a shared backing (it is in-memory today).
- **Queue and progress are in-memory.** A revision restart or crash loses `queued/running` job state and the progress feed; in-flight analyses must be re-submitted. Persisted runs are unaffected.
- **`az acr build` Unicode crash on Windows.** The local log streamer can die with `UnicodeEncodeError: charmap` while the server-side build succeeds. Verify with `az acr task list-runs -r $ACR --top 1 -o table` before assuming failure.
- **Docker Hub base-image rate limit** intermittently fails the CI `Build image in ACR` step (base images pulled from Docker Hub). Re-run the failed job or build in ACR directly (§3).
- **Auth is not touched by CI.** Only a Bicep template redeploy manages EasyAuth. A normal image deploy will neither fix nor break sign-in. If you "lose" the Entra secret you cannot read it back — you must reset (recreate) it and redeploy (§6).
- **Client secrets are write-only.** This applies to both the OpenAI Container App secret and the Entra client secret — there is no "show current value." Rotation always means set-new + apply.
- **Sandbox cold starts.** With `minReplicas: 0`, the first request after idle takes ~30–60 s. Not a failure; warm it before demos (§9).
- **Best-effort determinism only.** `OPENAI_SEED` reduces run-to-run drift but cannot guarantee identical output across model/infra updates, and does not settle genuine model-judgment differences on ambiguous drawings.
- **Reanalysis identity behind EasyAuth.** The reanalyze/analyze endpoints attribute the run to the signed-in engineer from the injected headers; anonymous scripted calls from outside an authenticated browser session will be challenged by the auth sidecar. Drive reruns from the authenticated UI or an authorized session.
- **Raw DB file copies can be torn.** Because of `noLock=1`, back up with the SQLite online-backup API (§7), not a naive file copy of a live, actively-written DB.